Trabajo Práctico II

SQL Avanzado: funciones, triggers, WITH, WITH RECURSIVE y funciones de ventana

MATERIA	Gestión Avanzada de Datos
CARRERA	Ingeniería en Sistemas de Información
FACULTAD	UTN
ALUMNO	Manuel Riedel
MOTOR	PostgreSQL 17
BASE DE DATOS	facultad
FECHA	21 de septiembre de 2026

Organización de la entrega

Cada ejercicio vive en su propio script y todos corren sobre la base **facultad**, esquema **public**. Los scripts son reejecutables: borran lo que van a crear antes de crearlo. Todas las salidas que aparecen en este documento son las que devolvió PostgreSQL 17 al correrlos de punta a punta, en el orden en que se listan.

`00-preparacion.sql`	Datos complementarios
`01-triggers-inscripciones.sql`	Ejercicio 1
`02-auditoria-inscripto.sql`	Ejercicio 2
`03-historial-precios.sql`	Ejercicio 3
`04-menor-inasistencia.sql`	Ejercicio 4
`05-salarios-desvio.sql`	Ejercicio 5
`06-ventana-salarios.sql`	Ejercicio 6
`07-grafo-contactos.sql`	Ejercicio 7
`08-arbol-dependencias.sql`	Ejercicio 8
`09-funcion-dependencia.sql`	Ejercicio 9
`10-relaciones-personas.sql`	Ejercicio 10, 10.1 y 10.2
`11-partes-productos.sql`	Ejercicio 11
`12-participacion-partes.sql`	Ejercicio 12
`13-ranking-partes.sql`	Ejercicio 13

Ajustes sobre la base de la cátedra

La base entregada no alcanzaba para resolver el práctico completo. Los cuatro ajustes que hubo que hacer son estos, todos concentrados en el script 00 salvo el último:

- La tabla **docente** estaba vacía y además no tenía columna de salario, que los ejercicios 5 y 6 necesitan. Se agregó `salario NUMERIC(12,2)` y se dieron de alta los ocho docentes sobre las personas 1 a 8, que ya existían y no estaban asignadas a ningún rol.
- La jerarquía de dependencia del ejercicio 8 se armó sobre la columna `docente.dependede`, que la base ya define como clave foránea a sí misma pero no tenía datos.
- La inscripción 21 estaba cargada sin ningún registro de asistencia. Se le cargaron tres clases con presencia completa, para que el empate del ejercicio 4 salga de datos reales y no de la ausencia de datos.
- La inscripción 17 estaba en PROMOCION con 8 horas de inasistencia, cuando el curso 4 permite 0 para promocionar y 4 para mantener la regularidad. Es decir, violaba las dos reglas que pide implementar el ejercicio

1. El script 01 normaliza de una vez las filas que ya estaban cargadas antes de que existieran los triggers; esa inscripción queda en LIBRE.

Las pruebas de los ejercicios 1 y 2 modifican inscripciones y asistencias, así que corren dentro de transacciones que terminan en `ROLLBACK`: la salida muestra el efecto de cada trigger y el juego de datos de la cátedra queda intacto. Por eso, al final de la corrida, `log_inscripto` queda vacía.

Datos complementarios

Alta de la columna `salario`, carga de los ocho docentes con su superior directo y de las asistencias que le faltaban a la inscripción 21. El árbol de dependencia queda con Gomez en la raíz, Smith y Perez debajo, y así hasta el tercer nivel.

SCRIPT 00-preparacion.sql

```
SET search_path TO public;

-- 0. Datos complementarios que el trabajo práctico necesita y la base no trae:
--    columna salario en docente, alta de los ocho docentes con su jerarquía de
--    dependencia, y asistencias de la inscripción 21, cargada sin registros.

ALTER TABLE docente ADD COLUMN IF NOT EXISTS salario NUMERIC(12,2);

TRUNCATE TABLE docente;

INSERT INTO docente (persona_id, legajo, dependede, salario) VALUES
    (1, 1001, NULL, 2400000.00),
    (2, 1002,    1, 1800000.00),
    (3, 1003,    1, 1750000.00),
    (4, 1004,    2, 1200000.00),
    (5, 1005,    2, 1150000.00),
    (6, 1006,    3, 1100000.00),
    (7, 1007,    4,  900000.00),
    (8, 1008,    4,  850000.00);

ALTER TABLE docente ALTER COLUMN salario SET NOT NULL;

INSERT INTO asistencia (inscripcion_id, fecha, horas) VALUES
    (21, DATE '2025-04-01', 4),
    (21, DATE '2025-05-01', 4),
    (21, DATE '2025-06-01', 4)
ON CONFLICT (inscripcion_id, fecha) DO NOTHING;

SELECT d.persona_id, p.apellidos, p.nombres, d.legajo, d.dependede, d.salario
FROM docente d
JOIN persona p ON p.persona_id = d.persona_id
ORDER BY d.persona_id;
```

SALIDA

```
psql:00-preparacion.sql:7: NOTICE:  column "salario" of relation "docente" already exists, skipping
 persona_id | apellidos | nombres  | legajo | dependede |  salario
------------+-----------+----------+--------+-----------+------------
          1 | Gomez     | Alan     |   1001 |           | 2400000.00
          2 | Smith     | Micaela  |   1002 |         1 | 1800000.00
          3 | Perez     | Albert   |   1003 |         1 | 1750000.00
          4 | Toma      | Alexis   |   1004 |         2 | 1200000.00
          5 | Jones     | Roberto  |   1005 |         2 | 1150000.00
          6 | Sea       | Susana   |   1006 |         3 | 1100000.00
          7 | Ada       | Martinez |   1007 |         4 |  900000.00
          8 | Keys      | Gonzalo  |   1008 |         4 |  850000.00
(8 rows)
```

1. Control de integridad de las inscripciones mediante triggers

El enunciado pide analizar qué evento controla cada regla y si conviene BEFORE o AFTER. Las tres respuestas son distintas:

- **a) Estado inicial REGULAR.** Trigger **BEFORE INSERT** sobre *inscripto*. Tiene que ser BEFORE porque la función pisa `NEW.estadocursada` y devuelve NEW: un trigger AFTER de nivel fila ignora el valor de retorno, así que no podría corregir la fila sin lanzar un UPDATE extra.
- **b) Validación de la promoción.** Trigger **BEFORE UPDATE** sobre *inscripto*, con cláusula WHEN que lo limita a las filas que pasan a PROMOCION sin estarlo antes. Es BEFORE porque la modificación hay que rechazarla, y la forma de rechazarla es un RAISE EXCEPTION antes de escribir. El WHEN evita entrar a la función en los UPDATE que no tienen nada que ver con la promoción.
- **c) Pase automático a LIBRE.** Acá el evento no es un UPDATE sobre *inscripto*: las inasistencias crecen cuando se cargan asistencias. Por eso el trigger va sobre *asistencia*, **AFTER INSERT OR UPDATE OR DELETE**, y actualiza *inscripto*. Es AFTER porque necesita que la fila de asistencia ya esté escrita para poder recalcular el total.

Las horas de inasistencia se calculan como la suma de `curso.horasxclase - asistencia.horas` sobre todas las clases de la inscripción. Ese cálculo lo usan las reglas b) y c), así que está factorizado en la función `fn_horas_inasistencia` en lugar de repetirlo en las dos.

La prueba recorre la inscripción 901 sobre el curso 3, que permite 4 horas para promocionar y 8 para seguir regular. Se da de alta pidiendo PROMOCION y queda REGULAR; con 4 horas de falta la promoción se acepta; con 8 se rechaza con excepción; con 12 el estado pasa solo a LIBRE.

SCRIPT 01-triggers-inscripciones.sql

```
SET search_path TO public;

-- 1. Horas de inasistencia acumuladas por una inscripción, usadas por los controles b) y c).

CREATE OR REPLACE FUNCTION fn_horas_inasistencia(p_inscripcion_id INTEGER)
RETURNS INTEGER
LANGUAGE sql STABLE AS $$
    SELECT COALESCE(SUM(c.horasxclase - a.horas), 0)::INTEGER
    FROM asistencia a
    JOIN inscripto i ON i.inscripcion_id = a.inscripcion_id
    JOIN curso     c ON c.curso_id       = i.curso_id
    WHERE a.inscripcion_id = p_inscripcion_id;
$$;

-- 1.a) Al dar de alta una inscripción, estadocursada debe quedar siempre en REGULAR,
--      cualquiera sea el valor indicado en el INSERT.

CREATE OR REPLACE FUNCTION fn_inscripto_estado_inicial()
RETURNS trigger
LANGUAGE plpgsql AS $$
BEGIN
    NEW.estadocursada := 'REGULAR';
    RETURN NEW;
END;
$$;

DROP TRIGGER IF EXISTS trg_inscripto_estado_inicial ON inscripto;

CREATE TRIGGER trg_inscripto_estado_inicial
BEFORE INSERT ON inscripto
FOR EACH ROW
EXECUTE FUNCTION fn_inscripto_estado_inicial();

-- 1.b) Sólo se acepta el paso a PROMOCION si las inasistencias no superan
--      horaspermpromocion del curso; en caso contrario se rechaza la modificación.

CREATE OR REPLACE FUNCTION fn_validar_promocion()
RETURNS trigger
LANGUAGE plpgsql AS $$
DECLARE
    v_inasistencias INTEGER;
```

```
    v_permitidas    SMALLINT;
BEGIN
    SELECT c.horaspermpromocion
      INTO v_permitidas
    FROM curso c
    WHERE c.curso_id = NEW.curso_id;

    v_inasistencias := fn_horas_inasistencia(NEW.inscripcion_id);

    IF v_inasistencias > v_permitidas THEN
        RAISE EXCEPTION
            'Inscripción %: no puede promocionar, acumula % horas de inasistencia y el curso permite %',
            NEW.inscripcion_id, v_inasistencias, v_permitidas;
    END IF;

    RETURN NEW;
END;
$$;

DROP TRIGGER IF EXISTS trg_validar_promocion ON inscripto;

CREATE TRIGGER trg_validar_promocion
BEFORE UPDATE ON inscripto
FOR EACH ROW
WHEN (NEW.estadocursada = 'PROMOCION'
      AND OLD.estadocursada IS DISTINCT FROM 'PROMOCION')
EXECUTE FUNCTION fn_validar_promocion();

-- 1.c) Cuando las inasistencias superan horaspermregular del curso,
--      la inscripción pasa automáticamente a LIBRE.

CREATE OR REPLACE FUNCTION fn_controlar_regularidad()
RETURNS trigger
LANGUAGE plpgsql AS $$
DECLARE
    v_inscripcion   INTEGER;
    v_inasistencias INTEGER;
    v_permitidas    SMALLINT;
BEGIN
    IF TG_OP = 'DELETE' THEN
        v_inscripcion := OLD.inscripcion_id;
    ELSE
        v_inscripcion := NEW.inscripcion_id;
    END IF;

    SELECT c.horaspermregular
      INTO v_permitidas
    FROM inscripto i
    JOIN curso     c ON c.curso_id = i.curso_id
    WHERE i.inscripcion_id = v_inscripcion;

    v_inasistencias := fn_horas_inasistencia(v_inscripcion);

    IF v_inasistencias > v_permitidas THEN
        UPDATE inscripto
           SET estadocursada = 'LIBRE'
         WHERE inscripcion_id = v_inscripcion
           AND estadocursada <> 'LIBRE';
    END IF;

    RETURN NULL;
END;
$$;

DROP TRIGGER IF EXISTS trg_controlar_regularidad ON asistencia;

CREATE TRIGGER trg_controlar_regularidad
AFTER INSERT OR UPDATE OR DELETE ON asistencia
FOR EACH ROW
EXECUTE FUNCTION fn_controlar_regularidad();

-- 1. Normalización de las inscripciones que ya estaban cargadas antes de existir
--    los triggers, para que la base arranque cumpliendo la regla c).

UPDATE inscripto i
   SET estadocursada = 'LIBRE'
```

```
  FROM curso c
 WHERE c.curso_id = i.curso_id
   AND i.estadocursada <> 'LIBRE'
   AND fn_horas_inasistencia(i.inscripcion_id) > c.horaspermregular;

-- 1. Pruebas

BEGIN;

INSERT INTO inscripto (inscripcion_id, anio, fecha, alumno, curso_id, estadocursada)
VALUES (901, 2025, DATE '2025-03-10', 107, 3, 'PROMOCION');

SELECT inscripcion_id, alumno, curso_id, estadocursada AS estado_tras_el_alta
FROM inscripto WHERE inscripcion_id = 901;

INSERT INTO asistencia (inscripcion_id, fecha, horas) VALUES (901, DATE '2025-04-07', 0);

SELECT fn_horas_inasistencia(901) AS inasistencias,
       c.horaspermpromocion, c.horaspermregular
FROM inscripto i JOIN curso c ON c.curso_id = i.curso_id
WHERE i.inscripcion_id = 901;

DO $$
BEGIN
    UPDATE inscripto SET estadocursada = 'PROMOCION' WHERE inscripcion_id = 901;
    RAISE NOTICE 'Promoción aceptada con % horas de inasistencia', fn_horas_inasistencia(901);
EXCEPTION WHEN OTHERS THEN
    RAISE NOTICE 'Promoción rechazada: %', SQLERRM;
END;
$$;

SELECT inscripcion_id, estadocursada AS estado FROM inscripto WHERE inscripcion_id = 901;

UPDATE inscripto SET estadocursada = 'REGULAR' WHERE inscripcion_id = 901;
INSERT INTO asistencia (inscripcion_id, fecha, horas) VALUES (901, DATE '2025-04-14', 0);

DO $$
BEGIN
    UPDATE inscripto SET estadocursada = 'PROMOCION' WHERE inscripcion_id = 901;
    RAISE NOTICE 'Promoción aceptada con % horas de inasistencia', fn_horas_inasistencia(901);
EXCEPTION WHEN OTHERS THEN
    RAISE NOTICE 'Promoción rechazada: %', SQLERRM;
END;
$$;

SELECT inscripcion_id, estadocursada AS estado FROM inscripto WHERE inscripcion_id = 901;

INSERT INTO asistencia (inscripcion_id, fecha, horas) VALUES (901, DATE '2025-04-21', 0);

SELECT fn_horas_inasistencia(901) AS inasistencias,
       i.estadocursada AS estado_automatico
FROM inscripto i WHERE i.inscripcion_id = 901;

ROLLBACK;

SELECT i.inscripcion_id, i.alumno, i.curso_id, i.estadocursada,
       fn_horas_inasistencia(i.inscripcion_id) AS inasistencias,
       c.horaspermregular, c.horaspermpromocion
FROM inscripto i
JOIN curso c ON c.curso_id = i.curso_id
ORDER BY i.inscripcion_id;
```

SALIDA

```
 inscripcion_id | alumno | curso_id | estado_tras_el_alta
----------------+--------+----------+---------------------
            901 |    107 |        3 | REGULAR
(1 row)

 inasistencias | horaspermpromocion | horaspermregular
---------------+--------------------+------------------
             4 |                  4 |                8
(1 row)

psql:01-triggers-inscripciones.sql:147: NOTICE:  Promoción aceptada con 4 horas de inasistencia
 inscripcion_id |  estado
----------------+-----------
            901 | PROMOCION
```

```
(1 row)

psql:01-triggers-inscripciones.sql:161: NOTICE:  Promoción rechazada: Inscripción 901: no puede promocionar,
        acumula 8 horas de inasistencia y el curso permite 4
 inscripcion_id | estado
----------------+---------
            901 | REGULAR
(1 row)

 inasistencias | estado_automatico
---------------+-------------------
            12 | LIBRE
(1 row)

 inscripcion_id | alumno | curso_id | estadocursada | inasistencias | horaspermregular | horaspermpromocion
----------------+--------+----------+---------------+---------------+------------------+--------------------
              1 |    101 |        1 | REGULAR       |             4 |                8 |                  4
              2 |    102 |        1 | PROMOCION     |             0 |                8 |                  4
              3 |    103 |        1 | LIBRE         |            12 |                8 |                  4
              4 |    104 |        1 | REGULAR       |             0 |                8 |                  4
              5 |    105 |        1 | LIBRE         |            12 |                8 |                  4
              6 |    101 |        2 | REGULAR       |             0 |                4 |                  0
              7 |    102 |        2 | REGULAR       |             0 |                4 |                  0
              8 |    103 |        2 | REGULAR       |             0 |                4 |                  0
              9 |    104 |        2 | REGULAR       |             0 |                4 |                  0
             10 |    105 |        2 | REGULAR       |             4 |                4 |                  0
             11 |    101 |        3 | REGULAR       |             4 |                8 |                  4
             12 |    102 |        3 | REGULAR       |             0 |                8 |                  4
             13 |    103 |        3 | REGULAR       |             0 |                8 |                  4
             14 |    104 |        3 | REGULAR       |             0 |                8 |                  4
             15 |    105 |        3 | REGULAR       |             0 |                8 |                  4
             16 |    101 |        4 | REGULAR       |             0 |                4 |                  0
             17 |    102 |        4 | LIBRE         |             8 |                4 |                  0
             18 |    103 |        4 | REGULAR       |             0 |                4 |                  0
             19 |    104 |        4 | REGULAR       |             0 |                4 |                  0
             20 |    105 |        4 | REGULAR       |             0 |                4 |                  0
             21 |    106 |        1 | REGULAR       |             0 |                8 |                  4
(21 rows)
```

2. Auditoría de modificaciones

`log_inscripto` guarda dos juegos completos de columnas, `ant_` y `nue_`, más la operación, `fechahora` con `DEFAULT NOW()` y `usuario` con `DEFAULT CURRENT_USER`. Con eso se conoce el estado de la inscripción antes y después de cada cambio.

Sobre lo que aportan OLD y NEW: en un INSERT sólo existe NEW, en un DELETE sólo existe OLD, y en un UPDATE existen las dos, OLD con la fila previa y NEW con la resultante. La función usa `TG_OP` para saber en cuál de los tres casos está y llenar únicamente las columnas que tienen sentido; en la salida se ve que la fila del INSERT tiene la mitad `ant_` en nulo y la del DELETE tiene en nulo la mitad `nue_`.

El trigger es AFTER INSERT OR UPDATE OR DELETE de nivel fila y devuelve NULL, que en un AFTER de fila es lo correcto porque el valor de retorno se descarta.

SCRIPT 02-auditoria-inscripto.sql

```
SET search_path TO public;

-- 2. Tabla de auditoría de inscripciones, con el estado anterior y el posterior
--    de cada modificación.
-- 2.a) fechahora TIMESTAMP con la fecha y hora del sistema por defecto.
-- 2.b) usuario, el usuario de base de datos que realizó la modificación.

DROP TABLE IF EXISTS log_inscripto;

CREATE TABLE log_inscripto (
    log_id             SERIAL,
    operacion          VARCHAR(10)   NOT NULL,
    fechahora          TIMESTAMP     NOT NULL DEFAULT NOW(),
    usuario            VARCHAR(50)   NOT NULL DEFAULT CURRENT_USER,
    ant_inscripcion_id INTEGER,
    ant_anio           SMALLINT,
    ant_fecha          DATE,
    ant_alumno         INTEGER,
    ant_curso_id       INTEGER,
    ant_estadocursada  estadocursada,
    nue_inscripcion_id INTEGER,
    nue_anio           SMALLINT,
    nue_fecha          DATE,
    nue_alumno         INTEGER,
    nue_curso_id       INTEGER,
    nue_estadocursada  estadocursada,
    CONSTRAINT pk_log_inscripto PRIMARY KEY (log_id),
    CONSTRAINT ck_log_inscripto_operacion CHECK (operacion IN ('INSERT', 'UPDATE', 'DELETE'))
);

-- 2. Trigger que registra automáticamente las modificaciones sobre inscripto.
--    OLD trae la fila previa al evento y NEW la resultante: en INSERT sólo hay NEW,
--    en DELETE sólo hay OLD y en UPDATE están ambas.

CREATE OR REPLACE FUNCTION fn_auditar_inscripto()
RETURNS trigger
LANGUAGE plpgsql AS $$
BEGIN
    IF TG_OP = 'INSERT' THEN
        INSERT INTO log_inscripto (operacion,
            nue_inscripcion_id, nue_anio, nue_fecha, nue_alumno, nue_curso_id, nue_estadocursada)
        VALUES (TG_OP,
            NEW.inscripcion_id, NEW.anio, NEW.fecha, NEW.alumno, NEW.curso_id, NEW.estadocursada);

    ELSIF TG_OP = 'UPDATE' THEN
        INSERT INTO log_inscripto (operacion,
            ant_inscripcion_id, ant_anio, ant_fecha, ant_alumno, ant_curso_id, ant_estadocursada,
            nue_inscripcion_id, nue_anio, nue_fecha, nue_alumno, nue_curso_id, nue_estadocursada)
        VALUES (TG_OP,
            OLD.inscripcion_id, OLD.anio, OLD.fecha, OLD.alumno, OLD.curso_id, OLD.estadocursada,
            NEW.inscripcion_id, NEW.anio, NEW.fecha, NEW.alumno, NEW.curso_id, NEW.estadocursada);

    ELSIF TG_OP = 'DELETE' THEN
        INSERT INTO log_inscripto (operacion,
            ant_inscripcion_id, ant_anio, ant_fecha, ant_alumno, ant_curso_id, ant_estadocursada)
        VALUES (TG_OP,
            OLD.inscripcion_id, OLD.anio, OLD.fecha, OLD.alumno, OLD.curso_id, OLD.estadocursada);
```

```
    END IF;

    RETURN NULL;
END;
$$;

DROP TRIGGER IF EXISTS trg_auditar_inscripto ON inscripto;

CREATE TRIGGER trg_auditar_inscripto
AFTER INSERT OR UPDATE OR DELETE ON inscripto
FOR EACH ROW
EXECUTE FUNCTION fn_auditar_inscripto();

-- 2. Pruebas

BEGIN;

INSERT INTO inscripto (inscripcion_id, anio, fecha, alumno, curso_id, estadocursada)
VALUES (902, 2025, DATE '2025-03-11', 108, 5, 'REGULAR');

UPDATE inscripto SET estadocursada = 'PROMOCION' WHERE inscripcion_id = 902;

UPDATE inscripto SET estadocursada = 'RECURSA' WHERE inscripcion_id = 902;

DELETE FROM inscripto WHERE inscripcion_id = 902;

SELECT log_id, operacion, usuario,
       ant_inscripcion_id, ant_estadocursada,
       nue_inscripcion_id, nue_estadocursada
FROM log_inscripto
ORDER BY log_id;

SELECT log_id, operacion, fechahora FROM log_inscripto ORDER BY log_id;

ROLLBACK;
```

SALIDA

```
 log_id | operacion | usuario | ant_inscripcion_id | ant_estadocursada | nue_inscripcion_id | nue_estadocursada
--------+-----------+---------+--------------------+-------------------+--------------------+-------------------
      1 | INSERT    | manuel  |                    |                   |                902 | REGULAR
      2 | UPDATE    | manuel  |                902 | REGULAR           |                902 | PROMOCION
      3 | UPDATE    | manuel  |                902 | PROMOCION         |                902 | RECURSA
      4 | DELETE    | manuel  |                902 | RECURSA           |                    |
(4 rows)

 log_id | operacion |         fechahora
--------+-----------+----------------------------
      1 | INSERT    | 2026-09-21 13:25:09.240637
      2 | UPDATE    | 2026-09-21 13:25:09.240637
      3 | UPDATE    | 2026-09-21 13:25:09.240637
      4 | DELETE    | 2026-09-21 13:25:09.240637
(4 rows)
```

3. Historial de cambios de precios

El registro tiene que generarse sólo cuando el precio efectivamente cambió, y para eso el trigger combina dos filtros que hacen cosas distintas:

- `AFTER UPDATE OF precio` hace que ni se evalúe el trigger si la sentencia no toca la columna precio. Es lo que pasa con el UPDATE que sólo cambia el nombre.
- `WHEN (OLD.precio IS DISTINCT FROM NEW.precio)` descarta las asignaciones que dejan el mismo valor. Se usa IS DISTINCT FROM y no <> porque con nulos la desigualdad devolvería NULL y el trigger no se dispararía.

La prueba hace cuatro UPDATE: uno que sube el precio de la rueda, uno que cambia el nombre, uno que reasigna el mismo precio y uno que sube el de la bujía. En el historial quedan dos filas, las dos que corresponden a cambios reales.

Los diez productos que carga este script son los mismos que usan los ejercicios 11, 12 y 13.

SCRIPT 03-historial-precios.sql

```
SET search_path TO public;

-- 3. Tablas productos e historial_precios, y trigger que registra cada cambio de precio
--    solamente cuando el precio efectivamente cambió.

DROP TABLE IF EXISTS partes;
DROP TABLE IF EXISTS historial_precios;
DROP TABLE IF EXISTS productos;

CREATE TABLE productos (
    id     SERIAL,
    nombre VARCHAR(100)   NOT NULL,
    precio NUMERIC(12,2)  NOT NULL,
    CONSTRAINT pk_productos PRIMARY KEY (id),
    CONSTRAINT ck_productos_precio CHECK (precio >= 0)
);

CREATE TABLE historial_precios (
    id_historial   SERIAL,
    id_producto    INTEGER       NOT NULL,
    precio_antiguo NUMERIC(12,2) NOT NULL,
    precio_nuevo   NUMERIC(12,2) NOT NULL,
    fecha          TIMESTAMP     NOT NULL DEFAULT NOW(),
    usuario        VARCHAR(50)   NOT NULL DEFAULT CURRENT_USER,
    CONSTRAINT pk_historial_precios PRIMARY KEY (id_historial),
    CONSTRAINT fk_historial_precios_productos
        FOREIGN KEY (id_producto) REFERENCES productos (id)
        ON UPDATE RESTRICT ON DELETE CASCADE
);

CREATE OR REPLACE FUNCTION fn_registrar_cambio_precio()
RETURNS trigger
LANGUAGE plpgsql AS $$
BEGIN
    INSERT INTO historial_precios (id_producto, precio_antiguo, precio_nuevo)
    VALUES (NEW.id, OLD.precio, NEW.precio);
    RETURN NULL;
END;
$$;

DROP TRIGGER IF EXISTS trg_historial_precios ON productos;

CREATE TRIGGER trg_historial_precios
AFTER UPDATE OF precio ON productos
FOR EACH ROW
WHEN (OLD.precio IS DISTINCT FROM NEW.precio)
EXECUTE FUNCTION fn_registrar_cambio_precio();

-- 3. Carga de productos usada también por los ejercicios 11, 12 y 13.

INSERT INTO productos (id, nombre, precio) VALUES
    ( 1, 'Automóvil',  38000000.00),
    ( 2, 'Rueda',        320000.00),
    ( 3, 'Llanta',       180000.00),
    ( 4, 'Neumático',    140000.00),
```

```
    ( 5, 'Motor',       9500000.00),
    ( 6, 'Pistón',       210000.00),
    ( 7, 'Bujía',         18000.00),
    ( 8, 'Válvula',       26000.00),
    ( 9, 'Tornillo',        900.00),
    (10, 'Chasis',      4200000.00);

SELECT setval('productos_id_seq', (SELECT MAX(id) FROM productos));

-- 3. Pruebas

UPDATE productos SET precio = 350000.00 WHERE id = 2;

UPDATE productos SET nombre = 'Rueda' WHERE id = 2;

UPDATE productos SET precio = 350000.00 WHERE id = 2;

UPDATE productos SET precio = 19500.00 WHERE id = 7;

SELECT h.id_historial, h.id_producto, p.nombre,
       h.precio_antiguo, h.precio_nuevo, h.usuario
FROM historial_precios h
JOIN productos p ON p.id = h.id_producto
ORDER BY h.id_historial;

SELECT COUNT(*) AS filas_en_historial FROM historial_precios;
```

SALIDA

```
psql:03-historial-precios.sql:41: NOTICE:  trigger "trg_historial_precios" for relation "productos" does not
        exist, skipping
 setval
--------
     10
(1 row)

 id_historial | id_producto | nombre | precio_antiguo | precio_nuevo | usuario
--------------+-------------+--------+----------------+--------------+---------
            1 |           2 | Rueda  |      320000.00 |    350000.00 | manuel
            2 |           7 | Bujía  |       18000.00 |     19500.00 | manuel
(2 rows)

 filas_en_historial
--------------------
                  2
(1 row)
```

4. Alumnos con menor cantidad de inasistencias

Tres CTE encadenadas: horas ausentes por inscripción, total por alumno y mínimo general. El mínimo se compara contra el total de cada alumno en lugar de usar `LIMIT 1`, que devolvería un solo alumno y el enunciado pide contemplar empates.

El LEFT JOIN a *asistencia* importa: una inscripción sin registros de asistencia tiene que contar cero y no desaparecer de la cuenta. El filtro del año va en el ON del LEFT JOIN y no en el WHERE, porque en el WHERE volvería a descartar esas filas.

Resultado: empatan los alumnos 104 y 106 con 0 horas. La segunda consulta muestra el total de los seis alumnos con cursada en 2025 como control.

SCRIPT 04-menor-inasistencia.sql

```
SET search_path TO public;

-- 4. Con WITH, alumno o alumnos que acumularon la menor cantidad total de horas
--    de inasistencia durante 2025, contemplando que varios compartan el mínimo.

WITH inasistencias_por_inscripcion AS (
    SELECT i.inscripcion_id,
           i.alumno,
           COALESCE(SUM(c.horasxclase - a.horas), 0) AS horas_ausente
    FROM inscripto i
    JOIN curso      c ON c.curso_id = i.curso_id
    LEFT JOIN asistencia a
           ON a.inscripcion_id = i.inscripcion_id
          AND EXTRACT(YEAR FROM a.fecha) = 2025
    WHERE i.anio = 2025
    GROUP BY i.inscripcion_id, i.alumno
),
inasistencias_por_alumno AS (
    SELECT ii.alumno,
           SUM(ii.horas_ausente) AS total_horas_ausente
    FROM inasistencias_por_inscripcion ii
    GROUP BY ii.alumno
),
minimo AS (
    SELECT MIN(total_horas_ausente) AS horas_minimas
    FROM inasistencias_por_alumno
)
SELECT ia.alumno AS alumno_id,
       p.apellidos,
       p.nombres,
       ia.total_horas_ausente AS horas_de_inasistencia
FROM inasistencias_por_alumno ia
JOIN alumno  al ON al.persona_id = ia.alumno
JOIN persona p  ON p.persona_id  = al.persona_id
CROSS JOIN minimo m
WHERE ia.total_horas_ausente = m.horas_minimas
ORDER BY ia.alumno;

-- 4. Detalle de control: totales de todos los alumnos con cursada en 2025.

WITH inasistencias_por_inscripcion AS (
    SELECT i.inscripcion_id,
           i.alumno,
           COALESCE(SUM(c.horasxclase - a.horas), 0) AS horas_ausente
    FROM inscripto i
    JOIN curso      c ON c.curso_id = i.curso_id
    LEFT JOIN asistencia a
           ON a.inscripcion_id = i.inscripcion_id
          AND EXTRACT(YEAR FROM a.fecha) = 2025
    WHERE i.anio = 2025
    GROUP BY i.inscripcion_id, i.alumno
)
SELECT ii.alumno AS alumno_id,
       p.apellidos,
       p.nombres,
       COUNT(*)                 AS cursos_inscriptos,
       SUM(ii.horas_ausente)    AS horas_de_inasistencia
FROM inasistencias_por_inscripcion ii
```

```
JOIN persona p ON p.persona_id = ii.alumno
GROUP BY ii.alumno, p.apellidos, p.nombres
ORDER BY horas_de_inasistencia, alumno_id;
```

SALIDA

```
 alumno_id | apellidos | nombres  | horas_de_inasistencia
-----------+-----------+----------+-----------------------
       104 | Alumno    | Alumno 4 |                     0
       106 | Alumno    | Alumno 6 |                     0
(2 rows)

 alumno_id | apellidos | nombres  | cursos_inscriptos | horas_de_inasistencia
-----------+-----------+----------+-------------------+-----------------------
       104 | Alumno    | Alumno 4 |                 4 |                     0
       106 | Alumno    | Alumno 6 |                 1 |                     0
       101 | Alumno    | Alumno 1 |                 4 |                     8
       102 | Alumno    | Alumno 2 |                 4 |                     8
       103 | Alumno    | Alumno 3 |                 4 |                    12
       105 | Alumno    | Alumno 5 |                 4 |                    16
(6 rows)
```

5. Salarios y desviación estándar

Una CTE calcula media y desviación una sola vez y el CROSS JOIN la pega a cada docente, de modo que los límites del intervalo aparecen en cada fila sin recalcularse. Se usa `STDDEV_SAMP`, la desviación muestral, que es la que aplica cuando se trata de una muestra y no del universo completo.

Con media 1.393.750 y desvío 538.143,30 el intervalo va de 855.606,70 a 1.931.893,30. Quedan seis de los ocho docentes: Gomez queda afuera por arriba y Keys por abajo, este último por poco más de cinco mil pesos. La segunda consulta lista a los ocho con una columna booleana para que se vea el corte.

SCRIPT 05-salarios-desvio.sql

```
SET search_path TO public;

-- 5. Con WITH, salario medio y desviación estándar de los docentes, y listado de
--    los docentes cuyo salario cae en el intervalo media ± desviación estándar.

WITH estadisticas AS (
    SELECT AVG(d.salario)             AS salario_medio,
           STDDEV_SAMP(d.salario)     AS desvio_estandar
    FROM docente d
    WHERE d.salario IS NOT NULL
)
SELECT d.persona_id      AS docente_id,
       d.legajo,
       p.apellidos,
       p.nombres,
       d.salario,
       ROUND(e.salario_medio, 2)                       AS salario_medio,
       ROUND(e.desvio_estandar, 2)                     AS desvio_estandar,
       ROUND(e.salario_medio - e.desvio_estandar, 2)   AS limite_inferior,
       ROUND(e.salario_medio + e.desvio_estandar, 2)   AS limite_superior
FROM docente d
JOIN persona p ON p.persona_id = d.persona_id
CROSS JOIN estadisticas e
WHERE d.salario IS NOT NULL
  AND d.salario BETWEEN e.salario_medio - e.desvio_estandar
                    AND e.salario_medio + e.desvio_estandar
ORDER BY d.salario DESC;

-- 5. Detalle de control: todos los docentes indicando si quedan dentro del intervalo.

WITH estadisticas AS (
    SELECT AVG(d.salario)         AS salario_medio,
           STDDEV_SAMP(d.salario) AS desvio_estandar
    FROM docente d
    WHERE d.salario IS NOT NULL
)
SELECT d.persona_id AS docente_id,
       p.apellidos,
       p.nombres,
       d.salario,
       d.salario BETWEEN e.salario_medio - e.desvio_estandar
                     AND e.salario_medio + e.desvio_estandar AS dentro_del_intervalo
FROM docente d
JOIN persona p ON p.persona_id = d.persona_id
CROSS JOIN estadisticas e
ORDER BY d.salario DESC;
```

SALIDA

```
 docente_id | legajo | apellidos | nombres  |  salario   | salario_medio | desvio_estandar | limite_inferior | limite_superior
------------+--------+-----------+----------+------------+---------------+-----------------+-----------------+-----------------
          2 |   1002 | Smith     | Micaela  | 1800000.00 |    1393750.00 |       538143.30 |       855606.70 |      1931893.30
          3 |   1003 | Perez     | Albert   | 1750000.00 |    1393750.00 |       538143.30 |       855606.70 |      1931893.30
          4 |   1004 | Toma      | Alexis   | 1200000.00 |    1393750.00 |       538143.30 |       855606.70 |      1931893.30
          5 |   1005 | Jones     | Roberto  | 1150000.00 |    1393750.00 |       538143.30 |       855606.70 |      1931893.30
          6 |   1006 | Sea       | Susana   | 1100000.00 |    1393750.00 |       538143.30 |       855606.70 |      1931893.30
          7 |   1007 | Ada       | Martinez |  900000.00 |    1393750.00 |       538143.30 |       855606.70 |      1931893.30
(6 rows)

 docente_id | apellidos | nombres  |  salario   | dentro_del_intervalo
------------+-----------+----------+------------+----------------------
          1 | Gomez     | Alan     | 2400000.00 | f
          2 | Smith     | Micaela  | 1800000.00 | t
          3 | Perez     | Albert   | 1750000.00 | t
          4 | Toma      | Alexis   | 1200000.00 | t
          5 | Jones     | Roberto  | 1150000.00 | t
          6 | Sea       | Susana   | 1100000.00 | t
          7 | Ada       | Martinez |  900000.00 | t
          8 | Keys      | Gonzalo  |  850000.00 | f
(8 rows)
```

6. Comparación de valores mediante funciones de ventana

Todo sale de una sola pasada sobre la tabla, sin una sola subconsulta. `AVG(salario) OVER ()` con la ventana vacía toma el conjunto entero de filas, así que la media aparece repetida en cada docente y la diferencia se calcula restando esa misma expresión.

Para la posición se usa `RANK() OVER (ORDER BY salario DESC)`. Acá no hay salarios repetidos y el ranking sale de 1 a 8, pero RANK es la elección correcta igual: si dos docentes cobraran lo mismo compartirían posición, que es lo razonable para un ranking de salarios.

SCRIPT 06-ventana-salarios.sql

```
SET search_path TO public;

-- 6. Con funciones de ventana y sin subconsultas: por cada docente, su salario,
--    el salario medio de todos, la diferencia contra esa media y la posición
--    que ocupa ordenando de mayor a menor.

SELECT d.persona_id AS docente_id,
       p.apellidos,
       p.nombres,
       d.salario,
       ROUND(AVG(d.salario) OVER (), 2)              AS salario_medio,
       ROUND(d.salario - AVG(d.salario) OVER (), 2)  AS diferencia_contra_la_media,
       RANK() OVER (ORDER BY d.salario DESC)         AS posicion
FROM docente d
JOIN persona p ON p.persona_id = d.persona_id
ORDER BY posicion;
```

SALIDA

```
 docente_id | apellidos | nombres  |  salario   | salario_medio | diferencia_contra_la_media | posicion
------------+-----------+----------+------------+---------------+----------------------------+----------
          1 | Gomez     | Alan     | 2400000.00 |    1393750.00 |                 1006250.00 |        1
          2 | Smith     | Micaela  | 1800000.00 |    1393750.00 |                  406250.00 |        2
          3 | Perez     | Albert   | 1750000.00 |    1393750.00 |                  356250.00 |        3
          4 | Toma      | Alexis   | 1200000.00 |    1393750.00 |                 -193750.00 |        4
          5 | Jones     | Roberto  | 1150000.00 |    1393750.00 |                 -243750.00 |        5
          6 | Sea       | Susana   | 1100000.00 |    1393750.00 |                 -293750.00 |        6
          7 | Ada       | Martinez |  900000.00 |    1393750.00 |                 -493750.00 |        7
          8 | Keys      | Gonzalo  |  850000.00 |    1393750.00 |                 -543750.00 |        8
(8 rows)
```

7. Rutas en un grafo

El grafo cargado tiene once aristas dirigidas sobre seis nodos, con dos ciclos: B y C se apuntan mutuamente, y D vuelve a A, lo que cierra el ciclo A-C-E-D-A. Entre A y D hay varios caminos alternativos, incluido el rodeo por F.

La protección contra los ciclos infinitos es el arreglo `camino`, que acumula los nodos ya recorridos: el término recursivo sólo avanza hacia destinos que cumplen `NOT c.nodo_destino = ANY (r.camino)`. Como en cada paso el arreglo crece y los nodos son finitos, la recursión termina sola, sin necesidad de un tope artificial de profundidad.

Salen siete caminos de A a D. El más corto es A -> C -> E -> D con 3 aristas y distancia 6. La última consulta lo aísla comparando contra el mínimo en lugar de usar `LIMIT 1`, para que un eventual empate de distancia devuelva los dos caminos.

SCRIPT 07-grafo-contactos.sql

```
SET search_path TO public;

-- 7. Tabla contactos y carga de un grafo con varios caminos entre A y D
--    y al menos un ciclo (B-C-B y A-C-E-D-A).

DROP TABLE IF EXISTS contactos;

CREATE TABLE contactos (
    nodo_origen  VARCHAR(10) NOT NULL,
    nodo_destino VARCHAR(10) NOT NULL,
    distancia    INTEGER     NOT NULL,
    CONSTRAINT pk_contactos PRIMARY KEY (nodo_origen, nodo_destino),
    CONSTRAINT ck_contactos_distancia CHECK (distancia > 0),
    CONSTRAINT ck_contactos_sin_bucle CHECK (nodo_origen <> nodo_destino)
);

INSERT INTO contactos (nodo_origen, nodo_destino, distancia) VALUES
    ('A', 'B',  5),
    ('A', 'C',  2),
    ('A', 'F', 10),
    ('B', 'C',  1),
    ('B', 'D',  4),
    ('C', 'B',  1),
    ('C', 'D',  7),
    ('C', 'E',  3),
    ('E', 'D',  1),
    ('F', 'D',  1),
    ('D', 'A',  6);

SELECT * FROM contactos ORDER BY nodo_origen, nodo_destino;

-- 7. Con WITH RECURSIVE, todos los caminos posibles de A hasta D indicando la
--    secuencia de nodos, la cantidad de aristas y la distancia total. El arreglo
--    camino guarda los nodos ya visitados y la condición NOT ... = ANY(camino)
--    impide volver a pasar por uno, evitando los ciclos infinitos.

WITH RECURSIVE rutas AS (
    SELECT c.nodo_origen,
           c.nodo_destino,
           ARRAY[c.nodo_origen, c.nodo_destino]::VARCHAR[] AS camino,
           1                                   AS aristas,
           c.distancia                         AS distancia_total
    FROM contactos c
    WHERE c.nodo_origen = 'A'

    UNION ALL

    SELECT r.nodo_origen,
           c.nodo_destino,
           r.camino || c.nodo_destino,
           r.aristas + 1,
           r.distancia_total + c.distancia
    FROM rutas r
    JOIN contactos c ON c.nodo_origen = r.nodo_destino
    WHERE NOT c.nodo_destino = ANY (r.camino)
)
SELECT array_to_string(camino, ' -> ') AS secuencia_de_nodos,
       aristas,
```

```
       distancia_total
FROM rutas
WHERE nodo_destino = 'D'
ORDER BY distancia_total, aristas;

-- 7. Camino de menor distancia entre A y D, admitiendo empates.

WITH RECURSIVE rutas AS (
    SELECT c.nodo_origen,
           c.nodo_destino,
           ARRAY[c.nodo_origen, c.nodo_destino]::VARCHAR[] AS camino,
           1                                  AS aristas,
           c.distancia                        AS distancia_total
    FROM contactos c
    WHERE c.nodo_origen = 'A'

    UNION ALL

    SELECT r.nodo_origen,
           c.nodo_destino,
           r.camino || c.nodo_destino,
           r.aristas + 1,
           r.distancia_total + c.distancia
    FROM rutas r
    JOIN contactos c ON c.nodo_origen = r.nodo_destino
    WHERE NOT c.nodo_destino = ANY (r.camino)
),
caminos_a_d AS (
    SELECT camino, aristas, distancia_total
    FROM rutas
    WHERE nodo_destino = 'D'
)
SELECT array_to_string(camino, ' -> ') AS camino_mas_corto,
       aristas,
       distancia_total
FROM caminos_a_d
WHERE distancia_total = (SELECT MIN(distancia_total) FROM caminos_a_d);
```

SALIDA

```
 nodo_origen | nodo_destino | distancia
-------------+--------------+-----------
 A           | B            |         5
 A           | C            |         2
 A           | F            |        10
 B           | C            |         1
 B           | D            |         4
 C           | B            |         1
 C           | D            |         7
 C           | E            |         3
 D           | A            |         6
 E           | D            |         1
 F           | D            |         1
(11 rows)

  secuencia_de_nodos   | aristas | distancia_total
-----------------------+---------+-----------------
 A -> C -> E -> D      |       3 |               6
 A -> C -> B -> D      |       3 |               7
 A -> B -> D           |       2 |               9
 A -> C -> D           |       2 |               9
 A -> B -> C -> E -> D |       4 |              10
 A -> F -> D           |       2 |              11
 A -> B -> C -> D      |       3 |              13
(7 rows)

 camino_mas_corto | aristas | distancia_total
------------------+---------+-----------------
 A -> C -> E -> D |       3 |               6
(1 row)
```

8. Árbol de dependencias jerárquicas

El árbol se recorre desde la raíz hacia abajo: el término base toma al docente con `dependede IS NULL` y le asigna nivel 0, y el término recursivo baja un nivel por cada salto. Como cada persona depende de una sola, la estructura es un árbol y el recorrido no se ramifica hacia arriba.

Aunque un árbol no puede tener ciclos, el arreglo de visitados está igual: si alguien cargara un dato inconsistente, la consulta se cortaría en lugar de colgarse.

La segunda versión agrega apellido y nombre de la persona y de su superior. El join al superior es un **LEFT JOIN**, que es lo que hace que la raíz aparezca en el resultado con las columnas del superior en nulo; con un join interno se perdería.

SCRIPT 08-arbol-dependencias.sql

```
SET search_path TO public;

-- 8. Con WITH RECURSIVE sobre persona y docente.dependede, nivel de cada persona
--    dentro del árbol: 0 la raíz, 1 quienes dependen de la raíz, y así sucesivamente.

WITH RECURSIVE jerarquia AS (
    SELECT d.persona_id,
           d.dependede,
           0                      AS nivel,
           ARRAY[d.persona_id]    AS visitados
    FROM docente d
    WHERE d.dependede IS NULL

    UNION ALL

    SELECT h.persona_id,
           h.dependede,
           j.nivel + 1,
           j.visitados || h.persona_id
    FROM jerarquia j
    JOIN docente h ON h.dependede = j.persona_id
    WHERE NOT h.persona_id = ANY (j.visitados)
)
SELECT persona_id AS idpersona,
       dependede,
       nivel
FROM jerarquia
ORDER BY nivel, idpersona;

-- 8. Misma jerarquía mostrando apellido y nombre de la persona y de aquella de la
--    que depende. La raíz aparece igual, con los datos del superior en nulo.

WITH RECURSIVE jerarquia AS (
    SELECT d.persona_id,
           d.dependede,
           0                      AS nivel,
           ARRAY[d.persona_id]    AS visitados
    FROM docente d
    WHERE d.dependede IS NULL

    UNION ALL

    SELECT h.persona_id,
           h.dependede,
           j.nivel + 1,
           j.visitados || h.persona_id
    FROM jerarquia j
    JOIN docente h ON h.dependede = j.persona_id
    WHERE NOT h.persona_id = ANY (j.visitados)
)
SELECT j.persona_id  AS idpersona,
       pe.apellidos  AS apellidos,
       pe.nombres    AS nombres,
       j.dependede   AS iddependede,
       ps.apellidos  AS apellidos_superior,
       ps.nombres    AS nombres_superior,
       j.nivel
FROM jerarquia j
```

```
JOIN persona pe      ON pe.persona_id = j.persona_id
LEFT JOIN persona ps ON ps.persona_id = j.dependede
ORDER BY j.nivel, j.persona_id;
```

SALIDA

```
 idpersona | dependede | nivel
-----------+-----------+-------
         1 |           |     0
         2 |         1 |     1
         3 |         1 |     1
         4 |         2 |     2
         5 |         2 |     2
         6 |         3 |     2
         7 |         4 |     3
         8 |         4 |     3
(8 rows)

 idpersona | apellidos | nombres  | iddependede | apellidos_superior | nombres_superior | nivel
-----------+-----------+----------+-------------+--------------------+------------------+-------
         1 | Gomez     | Alan     |             |                    |                  |     0
         2 | Smith     | Micaela  |           1 | Gomez              | Alan             |     1
         3 | Perez     | Albert   |           1 | Gomez              | Alan             |     1
         4 | Toma      | Alexis   |           2 | Smith              | Micaela          |     2
         5 | Jones     | Roberto  |           2 | Smith              | Micaela          |     2
         6 | Sea       | Susana   |           3 | Perez              | Albert           |     2
         7 | Ada       | Martinez |           4 | Toma               | Alexis           |     3
         8 | Keys      | Gonzalo  |           4 | Toma               | Alexis           |     3
(8 rows)
```

9. Función de dependencia entre personas

La función recorre la cadena de superiores de X hacia arriba y responde si Y aparece en algún punto de esa cadena, lo que cubre en una sola consulta la dependencia directa y la indirecta. Está escrita en SQL con la CTE recursiva adentro, como pide el enunciado.

Los cuatro casos dan lo esperado. El interesante es el último: con X e Y iguales devuelve falso, porque la cadena arranca en X y lo que se busca son los valores de `dependede`, y X nunca es superior de sí mismo. Es el resultado correcto: una persona no depende de sí misma.

La segunda consulta cruza todos los docentes contra todos y lista los catorce pares que cumplen la relación, que es la clausura transitiva del árbol.

SCRIPT 09-funcion-dependencia.sql

```
SET search_path TO public;

-- 9. Función que, dados los identificadores de dos personas X e Y, devuelve si X
--    depende directa o indirectamente de Y, resolviéndolo con una CTE recursiva.

CREATE OR REPLACE FUNCTION fn_depende_de(p_x INTEGER, p_y INTEGER)
RETURNS BOOLEAN
LANGUAGE sql STABLE AS $$
    WITH RECURSIVE cadena AS (
        SELECT d.persona_id,
               d.dependede,
               ARRAY[d.persona_id] AS visitados
        FROM docente d
        WHERE d.persona_id = p_x

        UNION ALL

        SELECT s.persona_id,
               s.dependede,
               c.visitados || s.persona_id
        FROM cadena c
        JOIN docente s ON s.persona_id = c.dependede
        WHERE NOT s.persona_id = ANY (c.visitados)
    )
    SELECT EXISTS (SELECT 1 FROM cadena c WHERE c.dependede = p_y);
$$;

-- 9. Pruebas de los cuatro casos pedidos.

SELECT 'X depende directamente de Y'   AS caso, 4 AS x, 2 AS y, fn_depende_de(4, 2) AS resultado
UNION ALL
SELECT 'X depende indirectamente de Y',      7,      1, fn_depende_de(7, 1)
UNION ALL
SELECT 'X no depende de Y',                  3,      2, fn_depende_de(3, 2)
UNION ALL
SELECT 'X e Y son la misma persona',         1,      1, fn_depende_de(1, 1);

-- 9. Matriz completa de dependencias entre todos los docentes.

SELECT x.persona_id AS x,
       y.persona_id AS y,
       fn_depende_de(x.persona_id, y.persona_id) AS x_depende_de_y
FROM docente x
CROSS JOIN docente y
WHERE x.persona_id <> y.persona_id
  AND fn_depende_de(x.persona_id, y.persona_id)
ORDER BY x.persona_id, y.persona_id;
```

SALIDA

```
             caso              | x | y | resultado
-------------------------------+---+---+-----------
 X depende directamente de Y   | 4 | 2 | t
 X depende indirectamente de Y | 7 | 1 | t
 X no depende de Y             | 3 | 2 | f
 X e Y son la misma persona    | 1 | 1 | f
(4 rows)

 x | y | x_depende_de_y
---+---+----------------
 2 | 1 | t
 3 | 1 | t
 4 | 1 | t
 4 | 2 | t
 5 | 1 | t
 5 | 2 | t
 6 | 1 | t
 6 | 3 | t
 7 | 1 | t
 7 | 2 | t
 7 | 4 | t
 8 | 1 | t
 8 | 2 | t
 8 | 4 | t
(14 rows)
```

10. Relaciones entre personas

Se reutiliza la tabla *persona* que ya existe en la base en lugar de crear otra igual, y se agregan *tipo_relacion* y *relacion*. *tipo_relacion* no es sólo una lista de valores: marca con `es_familiar` y `es_simetrica` cómo se comporta cada tipo, y el ejercicio 10.2 filtra por esa columna en vez de enumerar los tipos a mano.

Las relaciones se guardan una sola vez por par. Las consultas arman el grafo no dirigido con un `UNION` que agrega cada arista en los dos sentidos; se usa UNION y no UNION ALL para que un par cargado dos veces no duplique caminos.

Los datos son una familia de tres generaciones más un grupo de amistades. Hay ciclos en las dos partes: entre los amigos, 206-208-209-210-206; entre los familiares, 201 y 202 son cónyuges y ambos son padres de 203.

10.1. Distancia de amistad

Recorrido por niveles sobre las aristas de tipo AMIGO, quedándose con el mínimo cuando hay varios caminos. La condición `c.distancia > 0` es la que hace que una persona consigo misma devuelva 0 y no la longitud del ciclo que la trae de vuelta. Cuando no hay camino, `MIN` sobre el conjunto vacío devuelve NULL y el `COALESCE` lo convierte en 0, como pide el enunciado.

En los resultados, 206 y 211 dan 2 aunque haya también un camino de longitud 4 pasando por 208 y 209; y 206 con 212 da 0, porque lo único que los conecta es una relación de tipo CONOCIDO, que no cuenta como amistad.

10.2. Grado de parentesco

Misma estructura, pero el grafo se arma con los tipos marcados como familiares: padre o madre, hermanos y cónyuges. Abuelo y nieto dan 2 saltos, tía y sobrino también, y las relaciones directas dan 1. Sin relación familiar, 0.

SCRIPT 10-relaciones-personas.sql

```
SET search_path TO public;

-- 10. Esquema de relaciones entre personas. Se reutiliza la tabla persona ya
--     existente y se agregan tipo_relacion y relacion, que forman un grafo general
--     con múltiples caminos y ciclos.

DROP TABLE IF EXISTS relacion;
DROP TABLE IF EXISTS tipo_relacion;

CREATE TABLE tipo_relacion (
    tipo_relacion VARCHAR(15) NOT NULL,
    descripcion   VARCHAR(50) NOT NULL,
    es_familiar   BOOLEAN     NOT NULL,
    es_simetrica  BOOLEAN     NOT NULL,
    CONSTRAINT pk_tipo_relacion PRIMARY KEY (tipo_relacion)
);

CREATE TABLE relacion (
    persona_a     INTEGER     NOT NULL,
    persona_b     INTEGER     NOT NULL,
    tipo_relacion VARCHAR(15) NOT NULL,
    CONSTRAINT pk_relacion PRIMARY KEY (persona_a, persona_b, tipo_relacion),
    CONSTRAINT ck_relacion_sin_bucle CHECK (persona_a <> persona_b),
    CONSTRAINT fk_relacion_persona_a
        FOREIGN KEY (persona_a) REFERENCES persona (persona_id)
        ON UPDATE RESTRICT ON DELETE CASCADE,
    CONSTRAINT fk_relacion_persona_b
        FOREIGN KEY (persona_b) REFERENCES persona (persona_id)
        ON UPDATE RESTRICT ON DELETE CASCADE,
    CONSTRAINT fk_relacion_tipo
        FOREIGN KEY (tipo_relacion) REFERENCES tipo_relacion (tipo_relacion)
        ON UPDATE RESTRICT ON DELETE RESTRICT
);

INSERT INTO tipo_relacion (tipo_relacion, descripcion, es_familiar, es_simetrica) VALUES
    ('PADRE',    'persona_a es padre o madre de persona_b', TRUE,  FALSE),
    ('HERMANO',  'persona_a y persona_b son hermanos',      TRUE,  TRUE),
    ('CONYUGE',  'persona_a y persona_b son cónyuges',      TRUE,  TRUE),
    ('AMIGO',    'persona_a y persona_b son amigos',        FALSE, TRUE),
    ('CONOCIDO', 'persona_a y persona_b se conocen',        FALSE, TRUE);

-- 10. Personas y relaciones de prueba: una familia de tres generaciones y un grupo
```

```
--     de amistades que contiene un ciclo y varios caminos alternativos.

DELETE FROM persona WHERE persona_id BETWEEN 201 AND 212;

INSERT INTO persona (persona_id, tipodocumento, documento, apellidos, nombres) VALUES
    (201, 'DNI', '201', 'Alvarez',  'Jorge'),
    (202, 'DNI', '202', 'Alvarez',  'Marta'),
    (203, 'DNI', '203', 'Alvarez',  'Diego'),
    (204, 'DNI', '204', 'Alvarez',  'Laura'),
    (205, 'DNI', '205', 'Benitez',  'Silvia'),
    (206, 'DNI', '206', 'Alvarez',  'Nicolas'),
    (207, 'DNI', '207', 'Alvarez',  'Camila'),
    (208, 'DNI', '208', 'Castro',   'Pablo'),
    (209, 'DNI', '209', 'Diaz',     'Marina'),
    (210, 'DNI', '210', 'Esquivel', 'Tomas'),
    (211, 'DNI', '211', 'Ferrer',   'Ana'),
    (212, 'DNI', '212', 'Gimenez',  'Luis');

INSERT INTO relacion (persona_a, persona_b, tipo_relacion) VALUES
    (201, 202, 'CONYUGE'),
    (201, 203, 'PADRE'),
    (202, 203, 'PADRE'),
    (201, 204, 'PADRE'),
    (202, 204, 'PADRE'),
    (203, 204, 'HERMANO'),
    (203, 205, 'CONYUGE'),
    (203, 206, 'PADRE'),
    (205, 206, 'PADRE'),
    (203, 207, 'PADRE'),
    (205, 207, 'PADRE'),
    (206, 207, 'HERMANO'),
    (206, 208, 'AMIGO'),
    (208, 209, 'AMIGO'),
    (209, 210, 'AMIGO'),
    (210, 206, 'AMIGO'),
    (210, 211, 'AMIGO'),
    (211, 212, 'CONOCIDO');

SELECT r.persona_a, pa.apellidos AS apellido_a,
       r.persona_b, pb.apellidos AS apellido_b,
       r.tipo_relacion
FROM relacion r
JOIN persona pa ON pa.persona_id = r.persona_a
JOIN persona pb ON pb.persona_id = r.persona_b
ORDER BY r.tipo_relacion, r.persona_a, r.persona_b;

-- 10.1. Distancia de amistad entre dos personas: 1 los amigos directos, 2 el amigo
--       de un amigo, y 0 si no existe ningún camino de amistad. Se recorre el grafo
--       en ambos sentidos, se descartan los nodos ya visitados para evitar ciclos y
--       se devuelve la menor distancia cuando hay varios caminos.

CREATE OR REPLACE FUNCTION fn_distancia_amistad(p_a INTEGER, p_b INTEGER)
RETURNS INTEGER
LANGUAGE sql STABLE AS $$
    WITH RECURSIVE amistad AS (
        SELECT r.persona_a AS origen, r.persona_b AS destino
        FROM relacion r
        WHERE r.tipo_relacion = 'AMIGO'
        UNION
        SELECT r.persona_b, r.persona_a
        FROM relacion r
        WHERE r.tipo_relacion = 'AMIGO'
    ),
    camino AS (
        SELECT p_a AS nodo, 0 AS distancia, ARRAY[p_a] AS visitados

        UNION ALL

        SELECT a.destino,
               c.distancia + 1,
               c.visitados || a.destino
        FROM camino c
        JOIN amistad a ON a.origen = c.nodo
        WHERE NOT a.destino = ANY (c.visitados)
    )
    SELECT COALESCE(MIN(c.distancia), 0)
    FROM camino c
    WHERE c.nodo = p_b
      AND c.distancia > 0;
```

```
$$;

-- 10.2. Grado de parentesco entre dos personas: mínima cantidad de relaciones
--       familiares directas (padre/madre-hijo, hermanos y cónyuges) que hay que
--       recorrer, y 0 si no existe ninguna. Contempla ciclos y caminos múltiples.

CREATE OR REPLACE FUNCTION fn_grado_parentesco(p_a INTEGER, p_b INTEGER)
RETURNS INTEGER
LANGUAGE sql STABLE AS $$
    WITH RECURSIVE familia AS (
        SELECT r.persona_a AS origen, r.persona_b AS destino
        FROM relacion r
        JOIN tipo_relacion t ON t.tipo_relacion = r.tipo_relacion
        WHERE t.es_familiar
        UNION
        SELECT r.persona_b, r.persona_a
        FROM relacion r
        JOIN tipo_relacion t ON t.tipo_relacion = r.tipo_relacion
        WHERE t.es_familiar
    ),
    camino AS (
        SELECT p_a AS nodo, 0 AS grado, ARRAY[p_a] AS visitados

        UNION ALL

        SELECT f.destino,
               c.grado + 1,
               c.visitados || f.destino
        FROM camino c
        JOIN familia f ON f.origen = c.nodo
        WHERE NOT f.destino = ANY (c.visitados)
    )
    SELECT COALESCE(MIN(c.grado), 0)
    FROM camino c
    WHERE c.nodo = p_b
      AND c.grado > 0;
$$;

-- 10. Pruebas de distancia de amistad y grado de parentesco.

SELECT 'amigos directos'                AS caso, 206 AS a, 208 AS b, fn_distancia_amistad(206, 208) AS distancia
UNION ALL
SELECT 'amigo de un amigo',                    206,     209, fn_distancia_amistad(206, 209)
UNION ALL
SELECT 'camino más corto entre varios',        206,     211, fn_distancia_amistad(206, 211)
UNION ALL
SELECT 'sin camino de amistad',                206,     212, fn_distancia_amistad(206, 212)
UNION ALL
SELECT 'la misma persona',                     206,     206, fn_distancia_amistad(206, 206);

SELECT 'padre e hijo'                   AS caso, 203 AS a, 206 AS b, fn_grado_parentesco(203, 206) AS grado
UNION ALL
SELECT 'cónyuges',                             201,     202, fn_grado_parentesco(201, 202)
UNION ALL
SELECT 'hermanos',                             206,     207, fn_grado_parentesco(206, 207)
UNION ALL
SELECT 'abuelo y nieto',                       201,     206, fn_grado_parentesco(201, 206)
UNION ALL
SELECT 'tía y sobrino',                        204,     206, fn_grado_parentesco(204, 206)
UNION ALL
SELECT 'sin relación familiar',                206,     212, fn_grado_parentesco(206, 212);
```

SALIDA

```
 persona_a | apellido_a | persona_b | apellido_b | tipo_relacion
-----------+------------+-----------+------------+---------------
       206 | Alvarez    |       208 | Castro     | AMIGO
       208 | Castro     |       209 | Diaz       | AMIGO
       209 | Diaz       |       210 | Esquivel   | AMIGO
       210 | Esquivel   |       206 | Alvarez    | AMIGO
       210 | Esquivel   |       211 | Ferrer     | AMIGO
       211 | Ferrer     |       212 | Gimenez    | CONOCIDO
       201 | Alvarez    |       202 | Alvarez    | CONYUGE
       203 | Alvarez    |       205 | Benitez    | CONYUGE
       203 | Alvarez    |       204 | Alvarez    | HERMANO
       206 | Alvarez    |       207 | Alvarez    | HERMANO
       201 | Alvarez    |       203 | Alvarez    | PADRE
       201 | Alvarez    |       204 | Alvarez    | PADRE
       202 | Alvarez    |       203 | Alvarez    | PADRE
       202 | Alvarez    |       204 | Alvarez    | PADRE
       203 | Alvarez    |       206 | Alvarez    | PADRE
       203 | Alvarez    |       207 | Alvarez    | PADRE
       205 | Benitez    |       206 | Alvarez    | PADRE
       205 | Benitez    |       207 | Alvarez    | PADRE
(18 rows)

             caso              |  a  |  b  | distancia
-------------------------------+-----+-----+-----------
 amigos directos               | 206 | 208 |         1
 amigo de un amigo             | 206 | 209 |         2
 camino más corto entre varios | 206 | 211 |         2
 sin camino de amistad         | 206 | 212 |         0
 la misma persona              | 206 | 206 |         0
(5 rows)

         caso          |  a  |  b  | grado
-----------------------+-----+-----+-------
 padre e hijo          | 203 | 206 |     1
 cónyuges              | 201 | 202 |     1
 hermanos              | 206 | 207 |     1
 abuelo y nieto        | 201 | 206 |     2
 tía y sobrino         | 204 | 206 |     2
 sin relación familiar | 206 | 212 |     0
(6 rows)
```

11. Descomposición de productos en partes

La tabla *partes* referencia dos veces a *productos*, porque una parte es a su vez un producto que puede estar compuesto por otras. El CHECK `idproducto <> idparte` descarta el caso trivial de que algo se contenga a sí mismo.

La primera función devuelve las partes y subpartes con su nivel y la cantidad tal como está cargada. La segunda agrega `cantidad_real`, que multiplica la cantidad de cada nivel por la acumulada de los anteriores: es el caso del enunciado donde 4 unidades de A que llevan 3 de B cada una dan 12 de B.

Se ve en el automóvil. Lleva 4 ruedas y cada rueda 5 tornillos, o sea 20; el chasis aporta 20 más; los 4 pistones del motor llevan 3 cada uno, 12 más; y el motor lleva 2 tornillos propios. Total, 54 tornillos por automóvil, que es lo que devuelve la consulta agrupada.

Las dos funciones son SQL puro con WITH RECURSIVE adentro, y las dos llevan el arreglo de visitados para que una carga con un ciclo no cuelgue la consulta.

SCRIPT 11-partes-productos.sql

```
SET search_path TO public;

-- 11. Estructura de composición de productos en partes, con una cantidad arbitraria
--     de niveles: cada parte puede a su vez estar compuesta por otras partes.

DROP TABLE IF EXISTS partes;

CREATE TABLE partes (
    idproducto INTEGER NOT NULL,
    idparte    INTEGER NOT NULL,
    cantidad   INTEGER NOT NULL,
    CONSTRAINT pk_partes PRIMARY KEY (idproducto, idparte),
    CONSTRAINT ck_partes_cantidad CHECK (cantidad > 0),
    CONSTRAINT ck_partes_sin_bucle CHECK (idproducto <> idparte),
    CONSTRAINT fk_partes_producto
        FOREIGN KEY (idproducto) REFERENCES productos (id)
        ON UPDATE RESTRICT ON DELETE RESTRICT,
    CONSTRAINT fk_partes_parte
        FOREIGN KEY (idparte) REFERENCES productos (id)
        ON UPDATE RESTRICT ON DELETE RESTRICT
);

INSERT INTO partes (idproducto, idparte, cantidad) VALUES
    ( 1,  2,  4),
    ( 1,  5,  1),
    ( 1, 10,  1),
    ( 2,  3,  1),
    ( 2,  4,  1),
    ( 2,  9,  5),
    ( 5,  6,  4),
    ( 5,  7,  4),
    ( 5,  8,  8),
    ( 5,  9,  2),
    ( 6,  9,  3),
    (10,  9, 20);

SELECT pr.id AS idproducto, pr.nombre AS producto,
       pa.idparte, pp.nombre AS parte, pa.cantidad
FROM partes pa
JOIN productos pr ON pr.id = pa.idproducto
JOIN productos pp ON pp.id = pa.idparte
ORDER BY pa.idproducto, pa.idparte;

-- 11. Función SQL que, dado un idProducto, devuelve todas sus partes y subpartes
--     con el nivel en el que se encuentran y la cantidad requerida, usando
--     WITH RECURSIVE. El arreglo de visitados corta cualquier ciclo.

CREATE OR REPLACE FUNCTION fn_partes_de(p_idproducto INTEGER)
RETURNS TABLE (idproducto INTEGER, idparte INTEGER, nivel INTEGER, cantidad INTEGER)
LANGUAGE sql STABLE AS $$
    WITH RECURSIVE explosion AS (
        SELECT p.idproducto,
               p.idparte,
```

```
               1 AS nivel,
               p.cantidad,
               ARRAY[p.idproducto, p.idparte] AS visitados
        FROM partes p
        WHERE p.idproducto = p_idproducto

        UNION ALL

        SELECT p.idproducto,
               p.idparte,
               e.nivel + 1,
               p.cantidad,
               e.visitados || p.idparte
        FROM explosion e
        JOIN partes p ON p.idproducto = e.idparte
        WHERE NOT p.idparte = ANY (e.visitados)
    )
    SELECT e.idproducto, e.idparte, e.nivel, e.cantidad
    FROM explosion e
    ORDER BY e.nivel, e.idproducto, e.idparte;
$$;

-- 11. Extensión que calcula la cantidad real de cada componente necesaria para
--     fabricar una unidad del producto original, multiplicando la cantidad de cada
--     nivel por la acumulada de los niveles anteriores.

CREATE OR REPLACE FUNCTION fn_partes_explosionadas(p_idproducto INTEGER)
RETURNS TABLE (idproducto INTEGER, idparte INTEGER, nivel INTEGER,
               cantidad INTEGER, cantidad_real INTEGER)
LANGUAGE sql STABLE AS $$
    WITH RECURSIVE explosion AS (
        SELECT p.idproducto,
               p.idparte,
               1 AS nivel,
               p.cantidad,
               p.cantidad AS cantidad_real,
               ARRAY[p.idproducto, p.idparte] AS visitados
        FROM partes p
        WHERE p.idproducto = p_idproducto

        UNION ALL

        SELECT p.idproducto,
               p.idparte,
               e.nivel + 1,
               p.cantidad,
               e.cantidad_real * p.cantidad,
               e.visitados || p.idparte
        FROM explosion e
        JOIN partes p ON p.idproducto = e.idparte
        WHERE NOT p.idparte = ANY (e.visitados)
    )
    SELECT e.idproducto, e.idparte, e.nivel, e.cantidad, e.cantidad_real
    FROM explosion e
    ORDER BY e.nivel, e.idproducto, e.idparte;
$$;

-- 11. Pruebas

SELECT f.idproducto, pr.nombre AS producto,
       f.idparte, pp.nombre AS parte,
       f.nivel, f.cantidad
FROM fn_partes_de(1) f
JOIN productos pr ON pr.id = f.idproducto
JOIN productos pp ON pp.id = f.idparte
ORDER BY f.nivel, f.idproducto, f.idparte;

SELECT f.idproducto, f.idparte, pp.nombre AS parte,
       f.nivel, f.cantidad, f.cantidad_real
FROM fn_partes_explosionadas(1) f
JOIN productos pp ON pp.id = f.idparte
ORDER BY f.nivel, f.idproducto, f.idparte;

SELECT f.idparte, pp.nombre AS parte,
       SUM(f.cantidad_real) AS unidades_por_automovil
FROM fn_partes_explosionadas(1) f
```

```
JOIN productos pp ON pp.id = f.idparte
GROUP BY f.idparte, pp.nombre
ORDER BY unidades_por_automovil DESC, f.idparte;

SELECT f.idproducto, f.idparte, pp.nombre AS parte, f.nivel, f.cantidad_real
FROM fn_partes_explosionadas(5) f
JOIN productos pp ON pp.id = f.idparte
ORDER BY f.nivel, f.idparte;
```

SALIDA

```
psql:11-partes-productos.sql:6: NOTICE:  table "partes" does not exist, skipping
 idproducto | producto  | idparte |   parte   | cantidad
------------+-----------+---------+-----------+----------
          1 | Automóvil |       2 | Rueda     |        4
          1 | Automóvil |       5 | Motor     |        1
          1 | Automóvil |      10 | Chasis    |        1
          2 | Rueda     |       3 | Llanta    |        1
          2 | Rueda     |       4 | Neumático |        1
          2 | Rueda     |       9 | Tornillo  |        5
          5 | Motor     |       6 | Pistón    |        4
          5 | Motor     |       7 | Bujía     |        4
          5 | Motor     |       8 | Válvula   |        8
          5 | Motor     |       9 | Tornillo  |        2
          6 | Pistón    |       9 | Tornillo  |        3
         10 | Chasis    |       9 | Tornillo  |       20
(12 rows)

 idproducto | producto  | idparte |   parte   | nivel | cantidad
------------+-----------+---------+-----------+-------+----------
          1 | Automóvil |       2 | Rueda     |     1 |        4
          1 | Automóvil |       5 | Motor     |     1 |        1
          1 | Automóvil |      10 | Chasis    |     1 |        1
          2 | Rueda     |       3 | Llanta    |     2 |        1
          2 | Rueda     |       4 | Neumático |     2 |        1
          2 | Rueda     |       9 | Tornillo  |     2 |        5
          5 | Motor     |       6 | Pistón    |     2 |        4
          5 | Motor     |       7 | Bujía     |     2 |        4
          5 | Motor     |       8 | Válvula   |     2 |        8
          5 | Motor     |       9 | Tornillo  |     2 |        2
         10 | Chasis    |       9 | Tornillo  |     2 |       20
          6 | Pistón    |       9 | Tornillo  |     3 |        3
(12 rows)

 idproducto | idparte |   parte   | nivel | cantidad | cantidad_real
------------+---------+-----------+-------+----------+---------------
          1 |       2 | Rueda     |     1 |        4 |             4
          1 |       5 | Motor     |     1 |        1 |             1
          1 |      10 | Chasis    |     1 |        1 |             1
          2 |       3 | Llanta    |     2 |        1 |             4
          2 |       4 | Neumático |     2 |        1 |             4
          2 |       9 | Tornillo  |     2 |        5 |            20
          5 |       6 | Pistón    |     2 |        4 |             4
          5 |       7 | Bujía     |     2 |        4 |             4
          5 |       8 | Válvula   |     2 |        8 |             8
          5 |       9 | Tornillo  |     2 |        2 |             2
         10 |       9 | Tornillo  |     2 |       20 |            20
          6 |       9 | Tornillo  |     3 |        3 |            12
(12 rows)

 idparte |   parte   | unidades_por_automovil
---------+-----------+------------------------
       9 | Tornillo  |                     54
       8 | Válvula   |                      8
       2 | Rueda     |                      4
       3 | Llanta    |                      4
       4 | Neumático |                      4
       6 | Pistón    |                      4
       7 | Bujía     |                      4
       5 | Motor     |                      1
      10 | Chasis    |                      1
(9 rows)

 idproducto | idparte |  parte   | nivel | cantidad_real
------------+---------+----------+-------+---------------
          5 |       6 | Pistón   |     1 |             4
          5 |       7 | Bujía    |     1 |             4
          5 |       8 | Válvula  |     1 |             8
          5 |       9 | Tornillo |     1 |             2
          6 |       9 | Tornillo |     2 |            12
(5 rows)
```

12. Participación porcentual de las partes

`SUM(cantidad) OVER (PARTITION BY idproducto)` calcula el total de cada producto en la misma pasada, sin la consulta aparte que el enunciado pide evitar. El total se muestra en una columna propia para que se vea de dónde sale el porcentaje.

El automóvil suma 6 partes directas y las 4 ruedas representan el 66,67 %. El pistón y el chasis, que tienen una sola parte, dan 100 %.

SCRIPT 12-participacion-partes.sql

```
SET search_path TO public;

-- 12. Con una función de ventana, porcentaje que representa la cantidad de cada
--     parte sobre el total de partes directas requeridas por su producto, sin
--     consultar aparte el total de cada producto.

SELECT pa.idproducto,
       pr.nombre AS producto,
       pa.idparte,
       pp.nombre AS parte,
       pa.cantidad,
       SUM(pa.cantidad) OVER (PARTITION BY pa.idproducto) AS total_del_producto,
       ROUND(100.0 * pa.cantidad
             / SUM(pa.cantidad) OVER (PARTITION BY pa.idproducto), 2) AS porcentaje
FROM partes pa
JOIN productos pr ON pr.id = pa.idproducto
JOIN productos pp ON pp.id = pa.idparte
ORDER BY pa.idproducto, porcentaje DESC, pa.idparte;
```

SALIDA

```
 idproducto | producto  | idparte |   parte   | cantidad | total_del_producto | porcentaje
------------+-----------+---------+-----------+----------+--------------------+------------
          1 | Automóvil |       2 | Rueda     |        4 |                  6 |      66.67
          1 | Automóvil |       5 | Motor     |        1 |                  6 |      16.67
          1 | Automóvil |      10 | Chasis    |        1 |                  6 |      16.67
          2 | Rueda     |       9 | Tornillo  |        5 |                  7 |      71.43
          2 | Rueda     |       3 | Llanta    |        1 |                  7 |      14.29
          2 | Rueda     |       4 | Neumático |        1 |                  7 |      14.29
          5 | Motor     |       8 | Válvula   |        8 |                 18 |      44.44
          5 | Motor     |       6 | Pistón    |        4 |                 18 |      22.22
          5 | Motor     |       7 | Bujía     |        4 |                 18 |      22.22
          5 | Motor     |       9 | Tornillo  |        2 |                 18 |      11.11
          6 | Pistón    |       9 | Tornillo  |        3 |                  3 |     100.00
         10 | Chasis    |       9 | Tornillo  |       20 |                 20 |     100.00
(12 rows)
```

13. Ranking de partes por producto

El enunciado pide que dos partes con la misma cantidad ocupen la misma posición, y eso descarta de entrada `ROW_NUMBER()`, que numera sin repetir y rompe los empates por un criterio arbitrario. En la tabla comparativa se ve: al pistón y la bujía, que llevan 4 cada uno, ROW_NUMBER les da 2 y 3.

Entre las otras dos, la diferencia aparece en el motor, que tiene cantidades 8, 4, 4 y 2. `RANK()` le da al tornillo la posición 4, porque deja el hueco del empate; `DENSE_RANK()` le da la 3. Las dos cumplen la condición del enunciado, así que la elección es de criterio.

Se usó **DENSE_RANK()**: como lo que se pide es la posición de la parte dentro del producto, tiene más sentido que las posiciones sean consecutivas y que la tercera cantidad distinta sea la tercera posición. RANK sería preferible si lo que se quisiera informar fuese cuántas partes hay por encima de cada una.

SCRIPT 13-ranking-partes.sql

```
SET search_path TO public;

-- 13. Con funciones de ventana, por cada producto: parte, cantidad, porcentaje sobre
--     el total del producto y posición según la cantidad de mayor a menor, de modo
--     que dos partes con la misma cantidad compartan posición.

SELECT pa.idproducto,
       pr.nombre AS producto,
       pa.idparte,
       pp.nombre AS parte,
       pa.cantidad,
       ROUND(100.0 * pa.cantidad
             / SUM(pa.cantidad) OVER (PARTITION BY pa.idproducto), 2) AS porcentaje,
       DENSE_RANK() OVER (PARTITION BY pa.idproducto ORDER BY pa.cantidad DESC) AS posicion
FROM partes pa
JOIN productos pr ON pr.id = pa.idproducto
JOIN productos pp ON pp.id = pa.idparte
ORDER BY pa.idproducto, posicion, pa.idparte;

-- 13. Comparación de las tres funciones sobre los mismos datos: ROW_NUMBER numera
--     sin repetir y rompe los empates arbitrariamente, RANK los respeta pero deja
--     huecos y DENSE_RANK los respeta sin dejarlos, que es lo que pide el enunciado.

SELECT pa.idproducto,
       pa.idparte,
       pp.nombre AS parte,
       pa.cantidad,
       ROW_NUMBER() OVER (PARTITION BY pa.idproducto ORDER BY pa.cantidad DESC) AS row_number,
       RANK()       OVER (PARTITION BY pa.idproducto ORDER BY pa.cantidad DESC) AS rank,
       DENSE_RANK() OVER (PARTITION BY pa.idproducto ORDER BY pa.cantidad DESC) AS dense_rank
FROM partes pa
JOIN productos pp ON pp.id = pa.idparte
ORDER BY pa.idproducto, rank, pa.idparte;
```

SALIDA

```
 idproducto | producto  | idparte |   parte   | cantidad | porcentaje | posicion
------------+-----------+---------+-----------+----------+------------+----------
          1 | Automóvil |       2 | Rueda     |        4 |      66.67 |        1
          1 | Automóvil |       5 | Motor     |        1 |      16.67 |        2
          1 | Automóvil |      10 | Chasis    |        1 |      16.67 |        2
          2 | Rueda     |       9 | Tornillo  |        5 |      71.43 |        1
          2 | Rueda     |       3 | Llanta    |        1 |      14.29 |        2
          2 | Rueda     |       4 | Neumático |        1 |      14.29 |        2
          5 | Motor     |       8 | Válvula   |        8 |      44.44 |        1
          5 | Motor     |       6 | Pistón    |        4 |      22.22 |        2
          5 | Motor     |       7 | Bujía     |        4 |      22.22 |        2
          5 | Motor     |       9 | Tornillo  |        2 |      11.11 |        3
          6 | Pistón    |       9 | Tornillo  |        3 |     100.00 |        1
         10 | Chasis    |       9 | Tornillo  |       20 |     100.00 |        1
(12 rows)

 idproducto | idparte |   parte   | cantidad | row_number | rank | dense_rank
------------+---------+-----------+----------+------------+------+------------
          1 |       2 | Rueda     |        4 |          1 |    1 |          1
          1 |       5 | Motor     |        1 |          3 |    2 |          2
          1 |      10 | Chasis    |        1 |          2 |    2 |          2
          2 |       9 | Tornillo  |        5 |          1 |    1 |          1
          2 |       3 | Llanta    |        1 |          2 |    2 |          2
          2 |       4 | Neumático |        1 |          3 |    2 |          2
          5 |       8 | Válvula   |        8 |          1 |    1 |          1
          5 |       6 | Pistón    |        4 |          2 |    2 |          2
          5 |       7 | Bujía     |        4 |          3 |    2 |          2
          5 |       9 | Tornillo  |        2 |          4 |    4 |          3
          6 |       9 | Tornillo  |        3 |          1 |    1 |          1
         10 |       9 | Tornillo  |       20 |          1 |    1 |          1
(12 rows)
```